

Curso de Node y Express

Apuntes de contexto y programación

Elaborado y redactado por: Lihúén Villarreal

Índice de contenidos

Notas de contexto.....	2
Node.....	2
npm.....	2
Express.....	3
API REST.....	4
HTTP.....	4
Métodos HTTP.....	5
Códigos de estado HTTP.....	5
Programación con Node y Express.....	5
Servidor.....	5
Crear servidor con Node y Express.....	5
Routing.....	6
Routers.....	7
Ejecutar servidor.....	8
CRUD (Create, Read, Update, Delete).....	8
Método GET.....	8
Método POST.....	8
Método PUT.....	9
Método PATCH.....	10
Método DELETE.....	10

Notas de contexto

Node.js y **Express.js** son tecnologías muy usadas para crear aplicaciones web y APIs con JavaScript, especialmente del lado del servidor.

Node

Node.js es un **entorno de ejecución** que permite usar **JavaScript fuera del navegador**, es decir, en el servidor.

¿Para qué sirve?

- Crear **servidores web**
- Construir **APIs REST**
- Manejar **bases de datos**
- Crear aplicaciones en tiempo real (chats, juegos, notificaciones)
- Automatizar tareas (scripts, bots)

Características clave:

- **Asíncrono y no bloqueante** (muy rápido y eficiente)
- Usa el motor **V8** de Google (el mismo de Chrome)
- Incluye **npm**, un enorme gestor de paquetes
- Ideal para aplicaciones con muchas conexiones simultáneas

Ejemplo: un servidor que atiende miles de peticiones sin quedarse "bloqueado".

npm

Node Package Manager (Administrador de Paquetes de Node), o **npm**, es la herramienta oficial que se utiliza para:

- **Instalar, actualizar y eliminar** paquetes (librerías o dependencias) de JavaScript.
- **Gestionar dependencias** de un proyecto Node.js.
- **Ejecutar scripts** definidos en un proyecto.

Componentes clave:

- **Registro de npm**

- Es un repositorio en línea con millones de paquetes disponibles
- Sitio: npmjs.com
- **package.json**
 - Es un archivo que describe el proyecto
 - Define:
 - Nombre y versión del proyecto
 - Scripts
 - Dependencias
- **CLI (Command Line Interface)**
 - Se usa desde la terminal
 - Ejemplos:


```
npm init -y          # Crea un package.json
npm install express
# Instala Express y lo registra como dependencia
del proyecto.
```

Express

Express.js es un **framework** que funciona **encima de Node.js** y facilita mucho la creación de servidores y APIs.

Node es potente, pero algo “crudo”. Express lo hace **más simple y organizado**.

¿Para qué sirve?

- Crear **servidores web** fácilmente
- Definir **rutas** (`/usuarios`, `/productos`, etc.)
- Crear **APIs REST**
- Manejar peticiones HTTP (`GET`, `POST`, `PUT`, `DELETE`)
- Usar **middlewares** (para autenticación, logs, validaciones, etc.)

Características clave:

- Minimalista y flexible
- Basado en **middlewares** (funciones intermediarias que se ejecutan entre que el servidor recibe una petición y envía la respuesta)
- Muy usado para APIs backend

- Se integra con bases de datos, autenticación, etc.

El código con Express es más claro, corto y fácil de escalar.

API REST

Una **API REST** es una forma **estándar** de permitir que aplicaciones se comuniquen entre sí a través de la web, utilizando el protocolo **HTTP** de manera simple y estructurada.

- **API**: *Application Programming Interface*
→ Interfaz que permite que un programa use funcionalidades o datos de otro
- **REST**: *Representational State Transfer*
→ Estilo de arquitectura para diseñar servicios web

→ **API REST** = una API que sigue los principios REST

La API REST es un **servicio web** que:

- Expone **recursos** (usuarios, productos, pedidos, etc.)
- Usa **URLs** para identificarlos
- Utiliza los **métodos HTTP** estándar
- Intercambia datos normalmente representados en formatos **JSON** o **XML**

Ejemplo de API REST:

Supongamos un recurso "usuarios":

GET	/api/users	→ Obtener todos los usuarios
GET	/api/users/1	→ Obtener usuario con id 1
POST	/api/users	→ Crear usuario
PUT	/api/users/1	→ Actualizar usuario con id 1
DELETE	/api/users/1	→ Eliminar usuario con id 1

HTTP

HTTP es un **protocolo de comunicación** que permite el intercambio de información entre un **cliente** (por ejemplo, un navegador web) y un **servidor** a través de Internet.

- **HTTP**: *HyperText Transfer Protocol*
→ Protocolo de Transferencia de Hipertexto

HTTP define:

- Cómo se envían las **solicitudes (requests)**
- Cómo se retornan las **respuestas (responses)**
- Qué formato tienen los mensajes
- Qué métodos y códigos de estado se usan

HTTPS

- HTTP: datos en texto plano (no seguro)
 - HTTPS: HTTP + **cifrado TLS/SSL** (seguro)
- Hoy en día, casi todos los sitios usan **HTTPS**.

Métodos HTTP

GET	Obtener información
POST	Enviar datos
PUT	Actualizar datos
PATCH	Actualizar parcialmente
DELETE	Eliminar datos

Códigos de estado HTTP

200	OK (éxito)
201	Recurso creado
400	Solicitud incorrecta
401	No autorizado
404	Recurso no encontrado
500	Error del servidor

[Ver todos...](#)

Programación con Node y Express

Servidor

```
// -----  
// CREAR SERVIDOR CON NODE Y EXPRESS  
// -----  
  
// En la carpeta del proyecto, abrir terminal e ingresar:  
// npm init  
// Esto inicializa el paquete. Completar los datos que pide o saltar  
pasos con enter.
```

```

// Los datos no ingresados adquieren valor por defecto. Para
cambiarlos, editar archivo package.json

// En terminal, ingresar:
// npm install express
// Para instalar Express y sus dependencias.

// Importar Express:
const express = require('express');
const app = express();

// Importar objeto para simular base de datos:
const { productList } = require('./productList.js');

// Habilitar escucha del servidor en un puerto:
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`El servidor esta escuchando en el puerto
${PORT}...`);
});

// -----
// ROUTING
// -----

// Ruta principal (raiz, home o index):
app.get('/', (req, res) => {
  res.send('Respuesta del servidor.');
```

```

});

// Ruta del recurso sin modificaciones:
// app.get('/api/products', (req, res) => {
//   res.json(productList.products);
// });
```

```

// Para enviar respuestas en formato json:
// res.json();
// Es equivalente a:
// res.send();
```

```

// Ruta del recurso con parametro query:
```

```

app.get('/api/products', (req, res) => {

  // Si la ruta incluye el parametro query:
  // Ejemplo: ?sort=name
  if (req.query.sort == 'name') {
    let results = productList.products;
    results = results.sort(function (a, b) {
      let nameA = a.name.toLowerCase();
      let nameB = b.name.toLowerCase();
      return (nameA < nameB) ? -1 : (nameA > nameB) ? 1 : 0;
    });
    return res.json(results);
  }

  // Si la ruta no incluye el parametro query:
  res.json(productList.products);
});

// Ruta con parametro de url:
app.get('/api/products/:id', (req, res) => {
  let id = req.params.id;
  let results = productList.products.filter(product => product.id ==
id);

  if (results.length == 0) {
    return res.status(404).send(`Product ID ${id} not found.`);
    // return res.status(404).end();
  }
  res.json(results);
});

// -----
// ROUTERS
// -----

// Para rutas largas que se repiten muchas veces:
const routerProducts = express.Router();
app.use('/api/products', routerProducts);
// Luego puedo reescribir:
// app.get('/api/products')
// Como:

```

```
// routerProducts.get('/')

// -----
// EJECUTAR SERVIDOR
// -----

// Para testear servidor, ingresar en terminal (cmd):
// nodemon server.js
// Y luego abrir en navegador:
// localhost:3000
// Esta es la ruta raiz, home o index.
```

CRUD (Create, Read, Update, Delete)

```
// Metodos HTTP:
// Get - Leer
// Post - Crear
// Put, Patch - Actualizar
// Delete - Eliminar

// Para manejar las solicitudes con json,
// incluir middleware antes del routing:
app.use(express.json());

// -----
// METODO GET
// -----

// Manejo de solicitud por metodo get:
app.get('/api/products', (req, res) => {
  res.json(productList.products);
});

// Solicitud: ingresar a la url indicada en la ruta.
// http://localhost:3000/api/products

// -----
// METODO POST
// -----

// Manejo de solicitud por metodo post:
app.post('/api/products', (req, res) => {
```



```

    let newProduct = req.body;
    productList.products.push(newProduct);
    res.json(productList.products);
  });

// Solicitud:
// POST http://localhost:3000/api/products HTTP/1.1
// Content-Type: application/json
// {
//   "id": 50921,
//   "name": "Chevrolet Onix Joy",
//   "description": "Generación 2019, variedad de colores. Motor
1.0, ideal para ciudad.",
//   "cost": 13500,
//   "currency": "USD",
//   "soldCount": 14,
//   "image": "img/prod50921_1.jpg"
// }

// -----
// METODO PUT
// -----

// Manejo de solicitud por metodo put:
app.put('/api/products/:id', (req, res) => {
  let id = req.params.id;
  let updatedProduct = req.body;
  let index = productList.products.findIndex(product => product.id
== id);

  if (index >= 0) {
    productList.products[index] = updatedProduct;
  }
  res.json(productList.products);
});

// Solicitud:
// PUT http://localhost:3000/api/products/50921 HTTP/1.1
// Content-Type: application/json
// {
//   "id": 50921,

```

```

//   "name": "Chevrolet Onix Joy",
//   "description": "Generación 2019, variedad de colores. Motor
1.0, ideal para ciudad.",
//   "cost": 13500,
//   "currency": "USD",
//   "soldCount": 14,
//   "image": "img/prod50921_1.jpg"
// }

// -----
// METODO PATCH
// -----

// Manejo de solicitud por metodo patch:
app.patch('/api/products/:id', (req, res) => {
  let id = req.params.id;
  let updatedProperty = req.body;
  let index = productList.products.findIndex(product => product.id
== id);

  if (index >= 0) {
    let productToUpdate = productList.products[index];
    Object.assign(productToUpdate, updatedProperty);
  }
  res.json(productList.products);
});

// Solicitud:
// PATCH http://localhost:3000/api/products/50921 HTTP/1.1
// Content-Type: application/json
// {
//   "soldCount": 14,
// }

// -----
// METODO DELETE
// -----

// Manejo de solicitud por metodo delete:
app.delete('/api/products/:id', (req, res) => {
  let id = req.params.id;

```

```
    let index = productList.products.findIndex(product => product.id
== id);

    if (index >= 0) {
        productList.products.splice(index, 1);
    }
    res.json(productList.products);
});

// Solicitud:
// DELETE http://localhost:3000/api/products/50921 HTTP/1.1
```